

ELECTRONIC SYSTEMS 1

Laboratory – GNU Radio Companion and Python



Reinhard Feger, Werner Haselmayr

Winter Term 2025/2026

Institute for Communications Engineering and RF-Systems, nthfs.jku.at

Outline

Introduction

GNU Radio Companion

Python

INTRODUCTION



Introduction

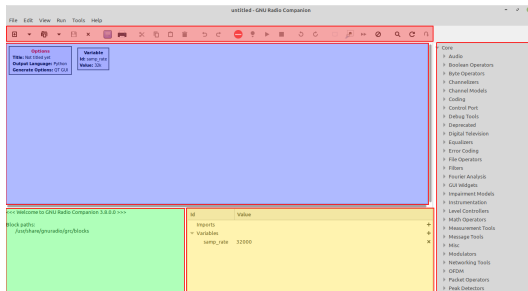
- This document serves as a reference for working with GNU Radio Companion
 - GNU Radio Companion is a software that allows to communicate with Software Defined Radio (SDR) systems
 - Universal software radio peripheral (USRP) is used as SDR device
 - Useful links
 - https://wiki.gnuradio.org/index.php/Guided_Tutorial_GRC
 - https://wiki.gnuradio.org/index.php/Main_Page
- This document also provides useful Python-Snippets for data analysis
 - **Hint:** Please study this document carefully to prevent major problems during the laboratory!

GNU RADIO COMPANION



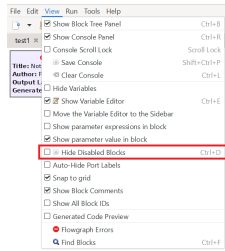
GNU Radio Companion – Overview

- Start GNU Radio Companion
- GNU Radio Companion is separated into five sections: Library, Toolbar, Terminal, Workspace and Variables



GNU Radio Companion – Basics

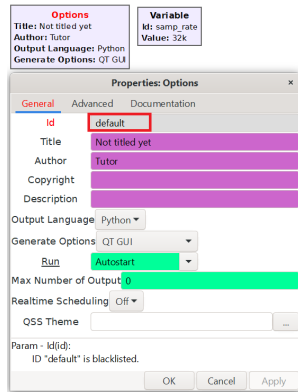
- All available blocks can be found in the *Library*-section
- Press `Ctrl+F` or the magnifying glass  in the toolbar to search for blocks
- Blocks can be placed by drag and drop
- Block properties can be opened by double-click
- Blocks can be temporarily enabled/disabled by right-clicking and selecting enable/disable in the drop-down menu **or** by selecting the block and press E/D to enable/disable the block



- **Note:** *Hide Disabled Blocks* under *View* must be **unchecked**, otherwise disabled blocks disappear

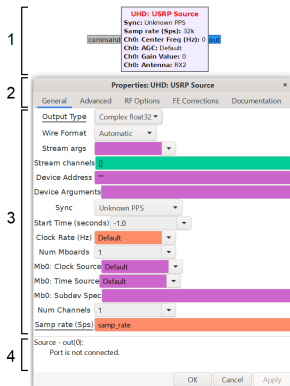
GNU Radio Companion – Basics

- The default Id (can be edited in the properties of the *Options*-block) must be changed to be able to run the file
- The program can be compiled with  or *F5*
- The program can be compiled and executed with  or *F6*
- The flow graph in the workspace is converted into a Python file by GNU Radio which is then executed



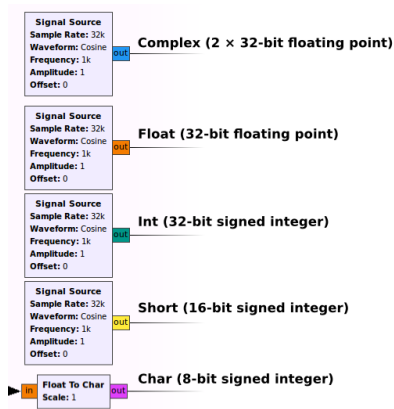
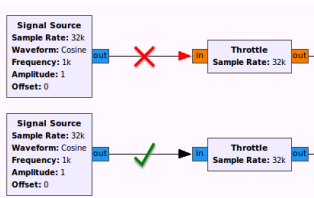
GNU Radio Companion – Block Properties

- Properties of blocks using the example of a *UHD: USRP Source*-block
 1. Block in the workspace
 2. Index tabs
 3. Settings
 4. Additional information of the block
- Important index tabs are *General*, *RF Options* and *Documentation* (information about its functionality)
- Setting values can also be expressed as a formula (e.g., $100 + \text{samp_rate} * 10$)



GNU Radio Companion – Data Types

- Data types are highlighted with different colors as shown on the right
- Only connections of the same data type are allowed
 - Data type of a block port can be changed in its properties



GNU Radio Companion – Variables

- Two basic variable blocks exist
- *Variable*
 - Static variable
 - Value is set in the block itself
 - Can be used in every block by its Id (here: *samp_rate*)
- *QT GUI Range*
 - Variable that can be changed during runtime
 - Variable value can be changed in the graphical interface (e.g., a slider)
 - Needs a *Default*-, *Start*-, *Stop*- and *Step*-value
 - Can be used in every block by its Id (here: *gain*)
 - Useful application: Gain, frequency, ...
- Values can be set in the form 32e3 (= 32000)

Variable
Id: samp_rate
Value: 32k

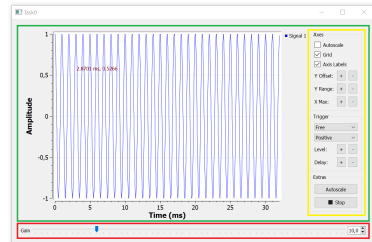
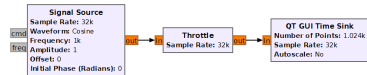
QT GUI Range
Id: gain
Label: Gain
Default Value: 10
Start: 0
Stop: 50
Step: 1

GNU Radio Companion – Data Visualization (1/2)

- *QT GUI Time Sink*

- Plots the time-domain signal of the connected signal
- Example (figure on the right side)

- *Signal Source*-block is connected to the *QT GUI Time Sink*-block
- Region highlighted in green shows the plot
- Region highlighted in red shows the slider of *QT GUI Range*-block (see slide 8)
- Region highlighted in yellow shows the control panel.
Can be activated through *QT GUI Time Sink*-block
→ *Properties* → *Config* → *Control Panel* → *Yes*

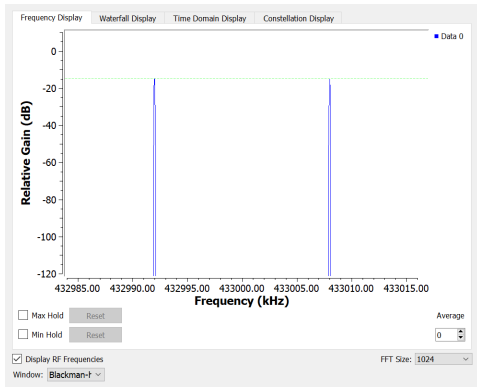


- *QT GUI Frequency Sink*

- Plots the frequency domain of the connected signal

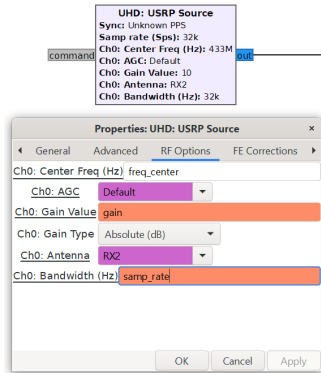
GNU Radio Companion – Data Visualization (2/2)

- *QT GUI Sink*
 - Combines both aforementioned plot types. Plot type can be selected via index tabs in the graphical interface
 - Use the checkbox *Display RF Frequencies* to plot the spectrum around a predefined center frequency (center frequency can be defined in the block's properties) or around 0.
 - Use the checkbox *Max Hold* to highlight the maximum amplitude at every frequency, which has occurred during the considered period



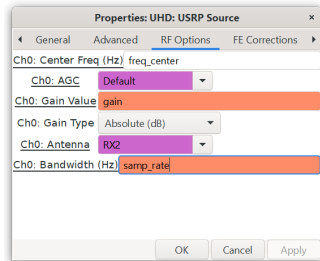
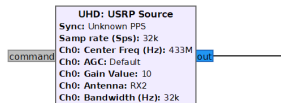
GNU Radio Companion – Capture Signals from Hardware (1/2)

- *UHD: USRP Source*-block enables the reception of complex signals from the hardware
- USRP contains (among others) a downconverter (two mixers) and analog-to-digital converters (ADCs)
- **Note:** The USRP shifts the frequency-band around the desired carrier down to zero. This could result in an asymmetric spectrum, which corresponds to a complex-valued time-domain signal.



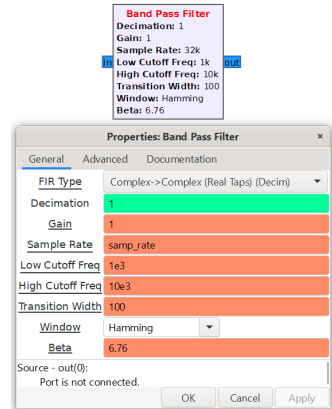
GNU Radio Companion – Capture Signal from Hardware (2/2)

- *Samp Rate* (cf. *General*-tab) and *Bandwidth* (cf. *RF Options*-tab) have to be set to *samp_rate*, which has the effect that the bandwidth at which the USRP records is equal to the value *samp_rate*
- *Center Freq* (cf. *RF Options*-tab) defines the mixing frequency of the USRP
- *Antenna* (cf. *RF Options*-tab) defines the port to which the antenna is connected (**hint:** when the USRP is reading data, a green LED lights up at the corresponding antenna port)



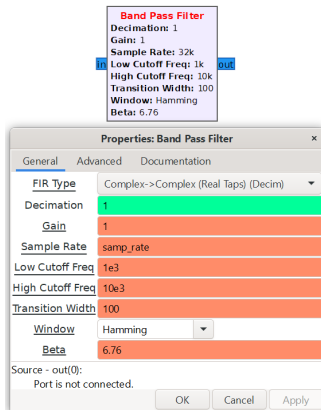
GNU Radio Companion – Filter (1/2)

- GNU Radio supports different filter types
 - High/low Pass
 - Band Pass
 - Band Reject
 - ...
- In the block properties the parameters *Low/High Cut Off Freq* and *Transition Width* (steepness of the filter) can be adjusted, which has to be adapted to the bandwidth requirements



GNU Radio Companion – Filter (2/2)

- To filter negative frequencies use the following setting: *General* → *FIR-Type* → *Complex* -> *Complex (Complex Taps) (Decim)*
- The value *Decimation* lowers the sample rate, which is useful if GNU Radio plots crash due to too much received data or too much open *QT sinks*

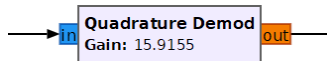


GNU Radio Companion – Quadrature Demod (1/2)

- *Quadrature Demod*-block^a demodulates 2-FSK, 4-FSK and GFSK modulated signals
- To receive a normalized signal use the following predefined formula in the block to calculate its *Gain* value

$$Gain = \frac{samp_rate}{2\pi \cdot fsk_deviation_hz},$$

where *fsk_deviation_hz* needs to be set to 1 Hz to obtain the frequency deviation as amplitude in the output signal



^awiki.gnuradio.org/index.php/Quadrature_Demod

GNU Radio Companion – Quadrature Demod (2/2)

- Functionality of *Quadrature Demod*-block with $fsk_deviation_hz = 1 \text{ Hz}$
 - The block delivers the phase difference between two complex-valued samples at discrete time instances n and $n - 1$ given by

$$\Delta\phi[n] = \phi[n] - \phi[n - 1],$$

where ϕ corresponds to the phase of a complex-valued number.

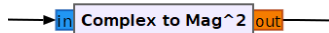
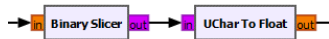
- Based on the phase difference the instantaneous frequency can be calculated by (see Lecture 1, slide 26-32)

$$f[n] = \frac{1}{2\pi} \frac{\Delta\phi[n]}{\Delta t} = \Delta\phi[n] \frac{f_{\text{sample}}}{2\pi},$$

with the sample rate ($samp_rate$) $f_{\text{sample}} = 1/\Delta t$.

GNU Radio Companion – Binary Slicer and Complex to Mag²

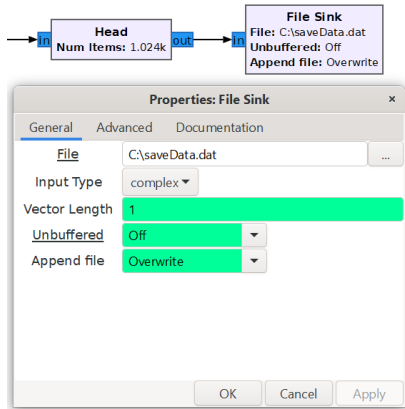
- The *Binary Slicer*-block is a simple threshold detector that maps the received signal of a 2-FSK modulation to 0 and 1
- The *UChar to Float*-block can be used to convert the output of the *Binary Slicer*-block to a Float
- The *Complex to Mag²*-block can be used to obtain the power of a (complex-valued) signal



$$x_{\text{out}} = |x_{\text{in}}|^2 = x_{\text{in}}^* x_{\text{in}}$$

GNU Radio Companion – Write to File

- For the signal analysis and to generate plots (for the report) it is required to store the collected data in files → *File Sink*-block
- Use the *Head*-block to save only a predefined amount of samples, rather than all measured samples
- Save the file using the *.dat* extension



PYTHON



Python – Read from File

- For the amount of samples defined in the *Head*-block (cf. *Num Items* on slide 18) a value of $1024e3$ is recommended. This value can be reduced if a *Decimation* greater than one is used in the *Filter*-block (e.g., $51.2e3$ for a decimation of 20).
- Python-Snippet to load data into Numpy

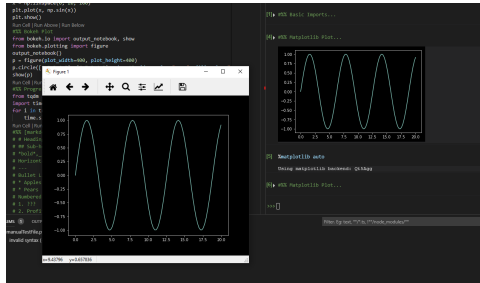
```
1 import numpy as np
2
3 filename = 'saveData.dat'
4 data = np.fromfile(filename, dtype=np.csingle)
5
6 N = 256*1024
7 data_clipped = data[0:N]
```

- Limit the viewed data by clipping, which can be achieved by changing the parameter N in the Python-Snippet above accordingly.
- **Note:** Depending on how much data is saved by the *File Sink*-block it may be hard to find the desired signals.

Python – Plots

- Python-Snippet to plot data with matplotlib

```
1 import matplotlib.pyplot as plt
2
3 plot1 = plt.figure(1)
4 plt.plot(np.real(data_clipped))
5 plt.xlabel("x-label")
6 plt.ylabel("y-label")
7 plt.title("title")
8 plt.show()
```



- Use `%matplotlib inline` or `%matplotlib qt` to switch between inline and windowed plots
- **Note:** Use `plt.figure()` to create separate windows, otherwise the curves will be plotted in the same figure